Rapport : gestion des livraisons

Projet réalisé en C. Le code est dans src/ et la compilation est expliquée dans le README.

1. Langage

J'ai choisi le C car on doit écrire nous-mêmes les structures et les algorithmes (tableau dynamique, tri, sac à dos). Ça permet de bien comprendre et justifier chaque choix.

2. Choix algorithmiques

n = nombre de colis, W = capacité du camion en kg.

Choix 1 : stockage des colis

Problème	Stocker beaucoup de colis sans connaître leur nombre à l'avance.
Solution	Tableau dynamique. Quand il est plein, on double sa taille avec realloc.
Temps	Ajout en O(1) en moyenne. Accès à un colis en O(1).
Espace	O(n).
Alternative	Liste chaînée : ajout facile, mais accès à un colis en O(n).
Justification	L'accès direct est utile pour le tri. Doubler la taille évite de recopier le tableau à chaque ajout.

Choix 2 : tri par poids, valeur ou priorité

Problème	Afficher les colis triés selon 3 critères, rapidement même avec 100 000 colis.
Solution	Tri fusion sur une copie du tableau. Le critère est passé en paramètre avec une fonction de comparaison.
Temps	O(n log n) dans tous les cas.
Espace	O(n) pour la copie et le tableau temporaire.
Alternative	Tri rapide ($O(n^2)$ au pire) ou tri à bulles ($O(n^2)$, beaucoup trop lent pour 100 000 colis).
Justification	Complexité garantie et une seule fonction de tri pour les 3 critères. Trier une copie garde l'ordre d'origine.

Choix 3 : préparation du camion

Problème	Choisir les colis qui rentrent dans le camion avec la plus grande valeur totale.
Solution	Programmation dynamique (sac à dos 0/1) : dp[w] = meilleure valeur pour une capacité w. Pour chaque colis, on garde le meilleur entre le prendre et ne pas le prendre.
Temps	O(n × W).
Espace	O(n × W) pour retrouver quels colis ont été pris.
Alternative	Glouton (meilleur rapport valeur/poids d'abord) : plus rapide mais pas toujours optimal. Avec 9 kg sur l'exemple, il trouve 25 au lieu de 32.
Justification	L'énoncé demande la meilleure valeur possible, et la programmation dynamique la trouve toujours.

Choix 4 : prise en compte de la priorité

Problème	Favoriser les colis prioritaires.
Solution	Priorité de 1 (faible) à 5 (forte). Score = valeur × priorité, puis même algorithme que le choix 3.
Temps	O(n × W).
Espace	O(n × W).
Alternative	Score = valeur + k × priorité, mais il faut choisir k.

Justification	Simple à comprendre : un colis de priorité 5 compte 5 fois sa valeur. Exemple avec 9 kg : sans priorité on charge 104 + 105, avec priorité 104 + 102.

3. Tests de performance

Capacité du camion : 50 kg. Temps en millisecondes.

n	Ajout	1000 recherches	Tri	Camion
100	0	0	0	0
1 000	0	1	0	0
10 000	1	4	1	0
100 000	9	238	11	5

Recherche par ID (O(n)) : c'est ce qui augmente le plus. Pour l'améliorer, on pourrait trier par ID et faire une recherche dichotomique en O(log n), ou utiliser une table de hachage.

Camion (O(n × W)) : rapide ici, mais la mémoire grossit avec n × W. Avec beaucoup de colis et une grande capacité, ça peut devenir trop lourd.

4. Questions

1. Choix le plus important

La programmation dynamique pour le camion. Le glouton ne trouve pas toujours la meilleure valeur (25 au lieu de 32 avec 9 kg), alors que la programmation dynamique donne toujours le résultat optimal. Elle sert aussi pour la priorité.

2. Problème avec plusieurs millions de colis

La recherche par ID, qui parcourt tout le tableau et est utilisée souvent. Et la mémoire du camion, qui grossit avec le nombre de colis multiplié par la capacité.

3. Vitesse ou mémoire

Plutôt la vitesse : le tri fusion utilise un tableau en plus pour rester en O(n log n), et le tableau des colis double sa taille pour que l'ajout reste rapide.

4. Meilleure stratégie pour le camion

Non, la programmation dynamique donne déjà la meilleure valeur possible. Pour la priorité, une autre formule donnerait un résultat différent, mais pas forcément meilleur.

5. Avec Python

Les algorithmes resteraient les mêmes. Je changerais juste la recherche par ID en utilisant un dictionnaire Python, beaucoup plus rapide.